

PROJET DOOM-LIKE P2P

COMPTE-RENDU D'AVANCEMENT : ITÉRATION 4

Équipe : Chailan Cyprian, Gros Geoffrey, Lounici Ilyes,
Ray Marcelin
18 février 2026

Sommaire

- **Rappel du projet et objectifs de l'itération**
- **Fonctionnalités actuelles**
 - **Ajout de l'automate de comportement des monstres**
 - **BSP* - Textures des murs**
 - **BSP - Sprites**

* (binary space partitioning ou BSP) est un système utilisé pour diviser l'espace en zones convexes. (wikipedia.fr)

Objectif de l'itération 4

But principal : Consolider le moteur de jeu en intégrant le rendu graphique optimisé et les premières interactions dynamiques

Travail Impératif :

- **Merge des prototypes des itérations précédentes**
- **Mise en place de l'automate et d'un environnement de test pour les déplacements des monstres**
- **Ajouter les collisions grâce à BSP**

Objectifs Optionnels :

- **Affichage de la texture des murs (BSP)**
- **Affichage des Sprites (BSP)**

L'Automate à états finis - IA des Monstres

Nous avons implémenté une “Intelligence Artificielle” basée sur un automate d'états finis pour gérer les prises de décision du monstre.

La boucle de jeu de l'IA évalue en permanence les conditions pour basculer entre :

- 1) **Attente** : Le monstre patiente un certain temps.
- 2) **Patrouille** : Déplacement régulier entre un point A et un point B.
- 3) **Poursuite** : S'active instantanément si une cible est détectée. Le monstre traque le joueur jusqu'à ce que la cible soit perdue ou capturée.

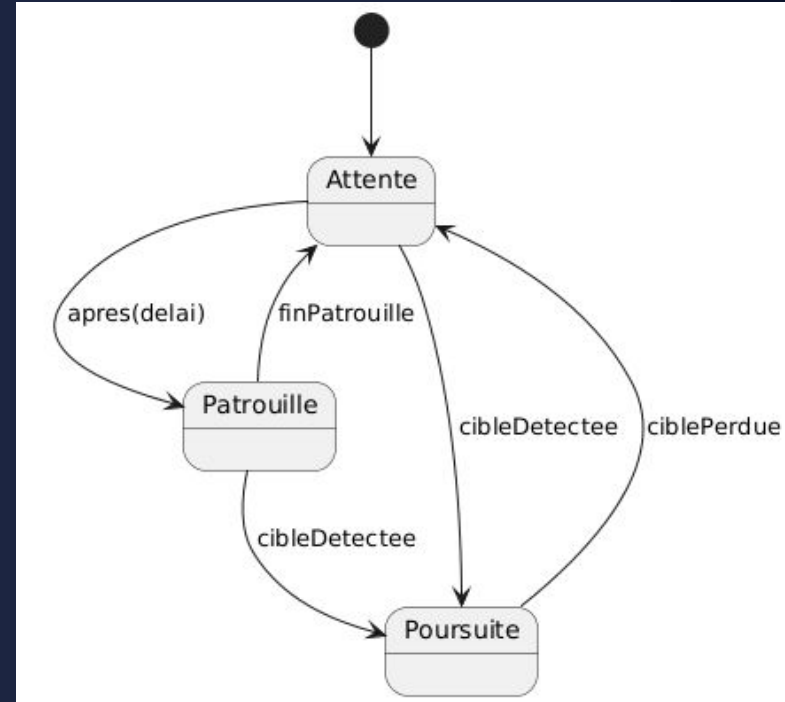


Diagramme dispo dans notre documentation
Automate.puml

Fonctionnement Automate

L'automate dicte *quoi* faire (l'intention), mais le *comment* (le mouvement réel) est géré par d'autres classes de notre architecture :

- Steering Behaviors : Pour le calcul de l'orientation fluide et la physique de déplacement (vitesse, direction)
- Algorithme RRT (Rapidly-exploring Random Tree) : Permet au monstre de calculer une route complexe pour contourner les murs (Pathfinding) lors de la poursuite ou de la patrouille

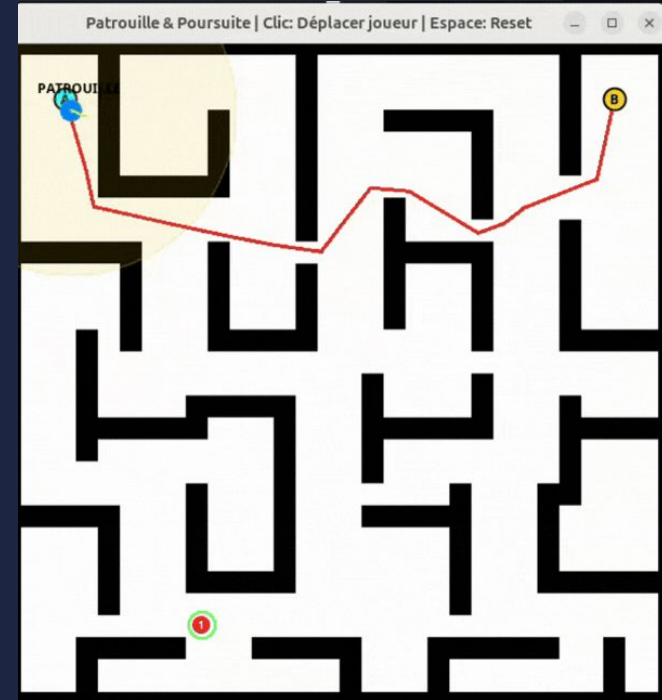
Rappel de la definition de pathfinding

- Selon le livre *Artificial Intelligence for Games*, le pathfinding (ou recherche de chemin) est défini comme la capacité d'une IA à **calculer un itinéraire approprié à travers un niveau de jeu** pour un personnage
- Concrètement, cela permet au monstre de calculer une route complète à l'avance pour contourner les murs et nous traquer efficacement, rendant ainsi le jeu plus difficile et réaliste.

L'Automate à états finis - Environnement de Simulation

Le débogage d'une IA dans un environnement 3D complet étant complexe, nous avons développé un environnement de test isolé (sandbox) en 2D

Cet environnement nous permet d'afficher la zone de détection du monstre, la generation du chemin par l'algorithme RRT et les transitions d'états de l'automate (Patrouille, Poursuite et Attente)



BSP - Texture des murs

OBJECTIF REPOUSSÉ

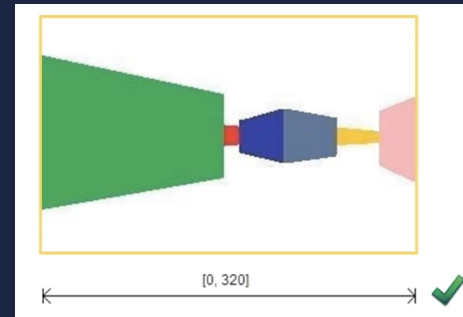
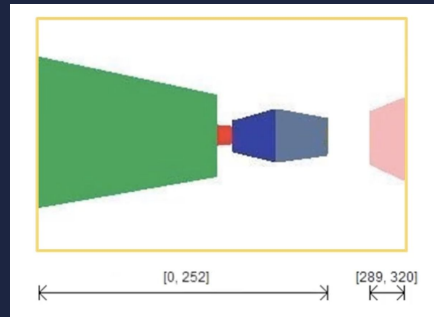
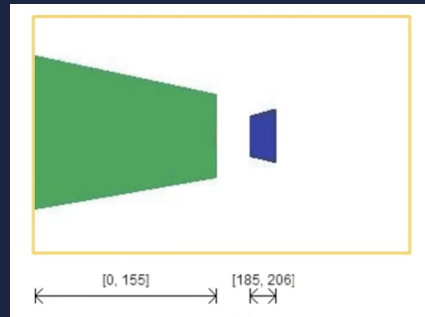
Suite à notre dernière réunion de suivi, l'ajout de texture des murs a été redéfini comme secondaire face aux enjeux d'architecture logicielle et de jouabilité.

- Implémentation du patron de conception Façade pour centraliser les systèmes (entrées, rendu, réseau)
- La logique mathématique de détection de collision des murs a été finalisée.
- Résolution du problème d'occultation des sprites par les segments de murs.
- L'intégration de l'automate d'états pour les monstres qui était une priorité.

BSP - Sprites

Parcours de l'arbre, rappel:

Front to back / occlusion à chaque nouveau mur ajouté / fin quand tout l'écran est rendu ou que tout l'arbre est parcourus.



<https://www.youtube.com/watch?v=yTRzfKh4Tg0>

BSP - Sprites, BSPParcours.java

L'idée est simple : au lieu d'avoir une liste de murs, on crée une liste d'objets. On insère les sprites au bon endroit parmi les murs déjà triés.

- 1) On récupère d'abord tous les murs visibles, comme avant.
- 2) Pour chaque Sprite, on compare sa position avec chaque mur.
- 3) La règle : Si un mur sépare le Joueur et le Sprite, alors le Sprite est "derrière" ce mur.
- 4) On décale l'index du Sprite dans la liste en fonction de ces obstacles.

BSP - Sprites, BSPParcours.java

Pseudo-code:

```
for (Sprite s : sprites) {  
    int index = 0;  
    for (Mur m : mursVisibles) {  
        if (murSépare(Joueur, Sprite)) {  
            index = position_apres_le_mur;  
        }  
    }  
    listeFinale.add(index, s);  
}
```

Exemple liste final objet:

```
[  
    [FourPoints, MurRef#12],  
    [FourPoints, MurRef#12],  
    [Sprite, null],  
    [FourPoints, MurRef#5],  
    [Sprite, null],  
    [FourPoints, MurRef#1],  
    [Sprite, null]  
]
```

BSP - Sprites, Renderer.java

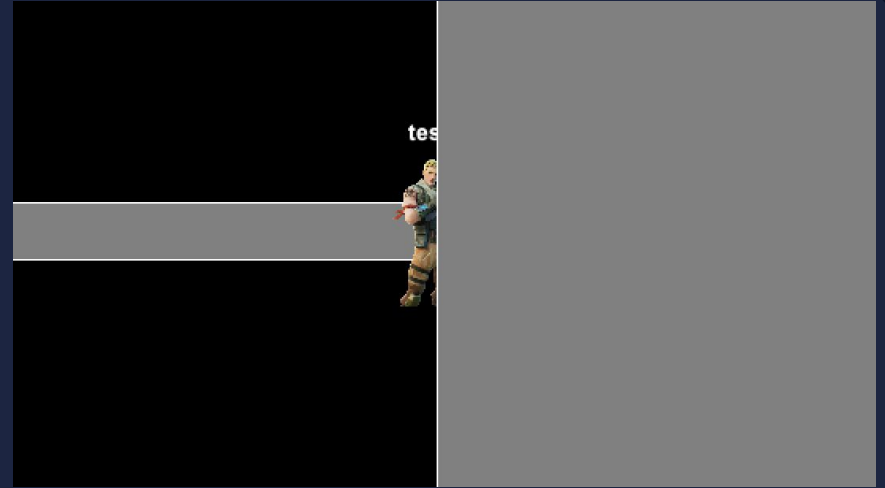
Le Renderer ne dessine plus seulement des polygones de murs, il accepte une liste d'objets : Mur ou Sprite.

- 1) On récupère la liste mixte (murs coupés + sprites positionnés).
- 2) On inverse l'ordre pour passer en Back-to-Front.
- 3) On dessine tout simplement : les murs de devant cachent les sprites de derrière naturellement.

BSP - Sprites, exemples



Rendu d'un monstre / joueur (affichage du pseudo en plus)



Occlusion d'un sprite, le mur devant "écrase" le rendu du joueur

BSP - Sprites, et DOOM (1993) ?

DOOM n'utilisait pas de "cropping" dynamique comme nous. Ils rendaient tout en Back-to-Front directement car les processeurs de l'époque gèrent mal les calculs de découpe complexe.

Notre choix technique :

- Conservation de l'existant : On garde notre parcours Front-to-Back car il gère déjà très bien l'occlusion des murs.
- Hybride : Une fois les murs "découpés" et les sprites placés, on repasse en Back-to-Front.
- Simplicité : Cette ré-inversion permet de gérer les sprites sans code d'occlusion supplémentaire (Z-Buffer complexe, etc).

Planning pour la prochaine itération

AUTOMATES :

- Réalisation d'un comportement de groupe pour les monstres
- Mise en place des monstres en réseau et en visuel dans notre jeu
- Débuggage éventuel et optimisation

BSP :

- Méthode pour récupérer les sprites visibles (pour comportement monstres)
- Textures des murs

Questions ?



Merci pour votre attention